SKILL LAB MILESTONE-1

A1. Identify Issues and list it with reasons

The issues are :

1.Redundancy- same movie can have many booking

2.All data is present in one table - thus it is difficult to manages everything like booking,payment.

3.1nf is violated as seat number can have multiple values.

4.2nf – user name depends on user and not on booking-id

5.3nf –theater location will depend on theatre name

6. And if suppose a new movie is released but , we can add only movie name we need to add all the others details as they are present in same table.

7. Same as previous one if we need to delete insertion of one attribute the whole row gets deleted .

A2. Normalize the Schema

```
CREATE TABLE BOOKINGS(
booking_id INT PRIMARY KEY,
booking_time INTEGER,
booking_date VARCHAR,
movie_id INT,
user_id INT,
THEATER_ID INT,
 FOREIGN KEY(user_id) REFERENCES USERS(user_id),
 FOREIGN KEY(movie_id) REFERENCES MOVIE(movie_id),
 FOREIGN KEY(THEATER_ID) REFERENCES THEATER(THEATER_ID)
);

CREATE TABLE USERS(
user_id INT PRIMARY KEY,
user_name VARCHAR,
user_email VARCHAR
);

CREATE TABLE MOVIE(
movie_id INT PRIMARY KEY,
movie_name VARCHAR,
MOVIE_DURATION  VARCHAR,
MOVIE_DATE VARCHAR,
PRICE INT,
RATING VARCHAR
);

CREATE TABLE MOVIE_TYPE(
TYPE_ID INT PRIMARY KEY,
movie_id INT,
```

```
LANGUAGE VARCHAR,
HERO VARCHAR,
HEROINE VARCHAR,
FOREIGN KEY(movie_id) REFERENCES MOVIE(movie_id)
);

CREATE TABLE THEATER(
THEATER_ID INT PRIMARY KEY,
THEATER_NAME VARCHAR,
THEATER_PLACE VARCHAR,
THEATER_TIMING VARCHAR
);

CREATE TABLE SEAT(
SEAT_id INT PRIMARY KEY,
NUMBER INT,
booking_id INT,
FOREIGN KEY(booking_id) REFERENCES BOOKINGS(booking_id)
);

CREATE TABLE PAYMENT(
PAYMENT_ID INT PRIMARY KEY,
TOTAL_PRICE INT,
PAYMENT_STATUS VARCHAR,
booking_id INT,
FOREIGN KEY(booking_id) REFERENCES BOOKINGS(booking_id)
);
  CREATE TABLE SEAT_TYPE(
   type_id INT PRIMARY KEY,
 type_name VARCHAR,
 extra_price INT,
 SEAT_id INT,
 FOREIGN KEY(seat_id) REFERENCES MOVIE(movie_id)
  );
```

1NF:

For seat_number there will be multiple inputs which will violate 1nf so creating a separate seat table.

2nf:

There partial dependency here as user_name will depend on user and even in theatre and movie.It is removed by moving creating separate tables for those and then linked to booking table.

3NF:

To only have details which are completely dependent on primary key of the table .here payment_status,seat_number shows transitive dependency.

A3. Assumptions

1. Each user is uniquely identified by user_id-assumes no two users share same identity

2. One booking is made by one user -a booking cannot belong to multiple users

3. A show represents a specific movie at specific interval and time-combines movie , theatre, show_time into one entity

4. A movie can have multiple shows in different theatre and time-theater location depends only on theatre

5. Each theatre has a fixed location-theater_location depends only on theatre

6. Each booking can include multiple seats-hence separate booking_seat table is required

7. Each seat is uniquely identified by seat_id -avoids duplication of seat information

8. Payment status is associated with each booking-payment is done per booking not seat

9. Total price is calculated per booking-based on number of seats and pricing rules

10. User email is unique for each user.

11. We can also include genre, rating, language, seat type, payment method….

SECTION B — DATABASE DESIGN FROM REQUIREMENTS

B1. Requirement Clarification

Q. What are tables to be created?

Q.What are attributes?

Q. Which is the primary key?

Q.Can user make multiple bookings?

Q. Where can we have 1:N relationships?

Q. Is phone_number single or multi-valued (need separate table)?

Q. Can a movie have multiple languages and genres

MOVIE

1. Can a movie have multiple languages and genres
2. Should language and genre be stored as separate tables or simple attributes
3. Can the same movie run in multiple cities and theaters simultaneously

THEATER

1.can a theatre have multiple screens

2.does each screen have a fixed number of seats and layout

3. are seat types

SHOW-

1.can the same screen run multiple shows in a day

2. can two shows overlap on the same screen

3. is pricing fixed per show or dynamic based on seat type

SEAT

1.is seat availability tracked per show or screen

2.can seat types have different prices within the same show

3.can a seat be temporily reserved before booking confirmation

Booking

1.can a user book multiple seats in one booking

2.can a booking include seats from fifferent shows

3.can bookings be cancelled or modified

USER

Can a user have multiple bookings over time

Should user history be stored permanently

Is user email unique for each user

LOCATION

Can a theatre exist in only one city

Should cities and locations be stored as separate entities

B2. Design

USERS

1. User_id – PRIMARY KEY
2. User_name
3. User_email

MOVIE

1. movie_id – PRIMARY KEY
2. movie_name

MOVIE_DETAILS

1. movie_id-foreign key
2. language
3. genre

CITY

1. city_id-primary key
2. city_name

THEATRE

1. theatre_id-primary key
2. theatre_name
3. city_id-foreign key

SCREEN

1. screen_id-primary key
2. theatre_id-foreign key
3. screen_name

SEAT_TYPE

1. seat_type_id – primary key
2. type_name

SEAT

1. seat_id-primary key
2. screen_id-foreign key
3. seat_type_id-foreign key
4. seat_number

SHOW

1. show_id-primary key
2. movie_id-foreign key
3. screen_id-foreign key
4. show_time

SHOW_PRICING

1. show_id-foreign key
2. seat_type_id-foreign key
3. price

BOOKING

1. booking_id-primary key
2. user_id-foreign key
3. show_id-foreign key
4. booking_date
5. total_price
6. payment_status

BOOKING_SEAT

1. booking_id-foreign key
2. seat_id-foreign key

SHOW_SEAT

1. show_id-foreign key
2. seat_id-foreign key
3. status

B3. Normalization

BOOKING

1. booking_id-primary key
2. show_id
3. booking_date
4. total_price
5. payment_status

change:removed user_id

BOOKING_USER

1. booking_id-foreign key
2. user_id-foreign key

BOOKING_USER_SEAT

1. booking_id-foreign key
2. user_id-foreign key
3. seat_id-foreign key

SECTION C — QUERIES AND TRANSACTIONS

C1. RETRIEVE ALL BOOKINGS FOR A USER WITHIN A DATARANGE

```
SELECT
b.booking_id,
m.movie_name,
t.theatre_name,
s.show_time,
se.seat_number,
FROM Bokkings b
JOIN Users u ON b.user_id = u.user_id
JOIN Shows s ON b.show_id = s.show_id
JOIN Movies m ON s.movie_id = s.movie_id
JOIN Screen sc ON s.screen_id = s.screen_id
JOIN Theatre t ON b.theatre_id = t.theatre_id
JOIN Booking_seat bs ON b.booking_id = bs.booking_id
JOIN Seat se   ON bs.seat_id = se.seat_id
WHERE u.user_id=1
AND b.booking_date BETWEEN ‘2026-01-01’ AND ‘2026-12-31’;
```

C2.MOST FREQUENTLY BOOKED MOVIE

```
SELECT
m.movie_name,
COUNT(b.booking_id) AS total_bookings
FROM Bookingd b
JOIN Shows s ON b.show_id = s.show_id
JOIN Movie m ON s.movie_id = m.movie_id
```

```
GROUP BY  m.movie_name
ORDER BY total_bookings DESC
LIMIT 1;
```

C3.SHOWS WITH TOTAL BOOKED AND AVAILABLE SEATS

```
SELECT
s.show_id,
m.movie_name,
s.show_time,
COUNT Shows s
JOIN Movie m ON s.movie_id = m.movie_id
JOIN Screen sc ON s.screen_id = sc.screen_id
JOIN Theatre t ON sc.theatre_id = t.theatre_id
JOIN Show_Seat ss ON s.show_id = ss.show_id
WHERE t.theatre_id =1
AND DATE(s.show_time) = ‘2026-04-28’
GROUP BY s.show_id, m.movie_name, s.show_time;
```

C4. TRANSACTION-BOOKING OPERATION

```
SELECT seat_id
FROM Show_Seat
WHERE show_id=101
AND seat_id IN (1,2,3)
AND status = ‘AVAILABLE’;
INSERT INTO Bookings(booking_id, user_id, show_id, booking_date, total_price,
payment_status)
VALUES(1001,1,101,NOW(), 500 .’Paid);
INSERT INTO Booking_Seat(booking_id, seat_id)
VALUES(1001,1),(1001,2),(1001,3);
UPDATE Show_Seat
SET status = ‘Booked’
WHERE show_id = 101
AND seat_id IN (1,2,3); AND THEN COMMIT
```